

Improving conceptual data models through iterative development



Tilman Zäschke^{a,*}, Stefania Leone^b, Tobias Gmünder^a, Moira C. Norrie^a

^a Institute for Information Systems, ETH Zurich CH-8092 Zurich, Switzerland

^b Semantic Information Research Laboratory, CS Department, USC Los Angeles, CA, 90089-0781, USA

ARTICLE INFO

Available online 10 July 2015

Keywords:

Database profiling
Evolvability
Object database
Agile development
Conceptual models
Model quality
Semantic verification

ABSTRACT

Agile methods promote iterative development with short cycles, where user feedback from the previous iteration is used to refactor and improve the current version. To facilitate agile development of information systems, this paper offers three contributions. First, we introduce the concept of *evolvability* as a model quality characteristic. Evolvability refers to the expected implications of future model refactorings, both in terms of complexity of the required database evolution algorithm and in terms of the expected volume of data to evolve. Second, we propose extending the agile development cycle by using database profiling information to suggest adaptations to the conceptual model to improve performance. For every software release, the database profiler identifies and analyses navigational access patterns, and proposes model optimisations based on data characteristics, access patterns and a cost–benefit model. Based on an experimental evaluation of the profiler we discuss why the quality of conceptual models can generally benefit from profiling and how performance measurements convey semantic information. Third, we discuss the flow of semantic information when developing and using information systems.

Beyond these contributions, we also make a case for using object databases in agile development environments. However, most of the presented concepts are also applicable to other database paradigms.

© 2015 Elsevier B.V. All rights reserved.

1. Introduction

Agile methods [1] are now widely accepted in software engineering. For the development of information systems, agile practices of early adoption and frequent software releases mean that, from the first release on, depending on the project settings, users may populate the database with application-specific data, see [2] for an example. One aim of our work is to exploit the presence of such real-world data at development time to optimise and verify the conceptual data model with respect to database performance. We therefore propose an approach that complements the feedback loop inherent to agile development practices with recommendations for conceptual model optimisations based on database profiling information.

Another aim of our work is to improve model quality beyond database performance. While model optimisation is traditionally applied to the physical or logical models of relational databases, we show how the conceptual model of an application can be optimised, resulting not only in faster applications but also in better models in terms of model quality and correctness. In terms of model quality, we also introduce a novel quality that describes how fast and easy a model and the corresponding database schema and data can be evolved. This notion of ‘evolvability’ is clearly useful in agile projects where schema and data evolution is frequent.

* Corresponding author.

E-mail addresses: zaeschke@inf.ethz.ch (T. Zäschke), stefania.leone@usc.edu (S. Leone), norrie@inf.ethz.ch (M.C. Norrie).

We use two main approaches to support the design and refactoring of data models while ensuring model quality. Our first approach uses standard static model analysis which, as the name suggests, is based on a static analysis of the application data model. Since static model analysis is inherently limited to the information specified in the model, it can only give a rough guidance for improving the quality of the conceptual model. However, while it cannot benefit from actual database usage statistics, it can still serve as a useful tool during the data model design process because the results are available in the model editor immediately after any model editing operation.

Our second approach to model optimisation is database profiling. Although it is traditionally advocated that conceptual models should be independent of the underlying database materialisation, we argue that database profiling can be exploited to improve the quality of a conceptual model based on insights into how a database is used and the data accessed. In addition to highlighting possible performance issues, it can also indicate whether the actual application usage correlates with the intended usage of the conceptual model. Comparing actual and intended usage provides opportunities for adapting and improving the model in accordance with evolving requirements and verifying whether the specified requirements match the actual usage. While it is already good practice to involve the end-users in the evaluation of the quality of conceptual models, database profiling has the advantage that it requires no additional effort from their side.

Using the profiling information, we can construct recommendations for refactorings in the data model. In this publication, we demonstrate the effectiveness of profiling and subsequently recommended refactorings. Based on the results from an example scenario, we argue that such refactorings will not only improve the application performance, but can also indicate and resolve semantic problems in the conceptual layer.

Finally, we take this thought a step further and discuss more generally the flow of semantic information when developers and database administrators (DBAs) construct an information system from user requirements and when this information system is eventually used by users. This results in some interesting considerations, for example that mapping a UML model to a relational data model and subsequent normalisation may effectively be seen as a weakness of the development process because it prunes the database data model of certain semantic information, such as inheritance information, and subjective understanding of the problem domain. However, our reasoning suggests that semantic information and subjective understanding are crucial for improving the quality of the conceptual model through profiling.

We focus here on object databases (ODBMSs) as the case has been made that they are well-suited to agile development [3] since they simplify the model and database evolution process. We support this view based on our own experience with a multi-terabyte real-world ODBMS [4]. The simplification of the development process follows from the fact that, in object databases, the conceptual model corresponds to the logical model and, unlike relational databases, there is no need for an object-relational-mapping layer. Changes to the conceptual model between frequent software releases require therefore only the change of the conceptual model, whereas relational databases (RDBMSs) also require adaptation of a separate logical model and the mapping layer between conceptual and logical model.

ODBMSs are especially well-suited to profiling of conceptual models because, in comparison, profiling the logical model in an RDBMS requires a reverse-mapping of the profiling results, which may not be trivial. Furthermore, we expect that profiling results from RDBMSs tend to be semantically less rich, because the process of normalisation in relational database models may remove semantic information.

In contrast to relational databases, ODBMSs support data access through navigation as well as queries. While query optimisation is typically supported through additional data structures such as indexes, navigation is reference-based and therefore tightly bound to the conceptual model. Our approach focuses on model optimisations based on profiling navigational data access paths. For every agile software release, we track and analyse the data access paths and data characteristics in order to propose conceptual model optimisations based on model refactoring patterns and evaluated with a cost–benefit model. The recommendations can be complemented with recommendations from query and static model analysis. We implemented our profiling framework as part of the ZooDB¹ object database and integrated it into AgileIS, a framework for model-driven agile information system development [4]. Recommendations for model optimisations resulting from database profiling are visualised as part of the graphical model editor. As proof of concept, we developed a research publication management system with data from DBLP [5] using our agile development framework together with the extended ZooDB and measured the performance improvements resulting from the recommended optimisations.

The contributions of this publication can be summarised as follows:

- We present the idea of *evolvability*.
- We demonstrate the concept of profiling conceptual models.
- We discuss the flow of semantic information through database usage.

This article extends [6] to show not only how agile development practices can be improved to include conceptual modelling and how object databases support agile development practices, but also what profiling can contribute to model quality beyond model performance and how evolvability can be derived from model analysis.

In Section 2, we discuss related research and then present our approach in Section 3. This is followed by a more detailed discussion of static model analysis in Section 4. Section 5 introduces database profiling with Section 5.1 discussing navigational access paths, while the refactoring patterns and associated cost–benefit models are presented in Section 5.2. In Sections 5.3 and 5.4, we detail the collection of profiling data and how it is brought to the developer in a model-driven development environment. A case study

¹ <https://github.com/tzaeschke/zoodb>.

used to evaluate the approach is presented in Section 5.5. After that, we take a step back and discuss some more general ideas about how semantic information flows during the development of information systems in Section 6. In Section 7 we discuss the advantages of ODBMSs in agile development environments. We then discuss our approach and give an outlook in Section 8. Concluding remarks are given in Section 9.

2. Background

The majority of research in database optimisation focuses on optimising the logical or physical layer of relational databases with respect to queries [7], either manually or automatically. Self-tuning databases [8] perform automated database refactorings, for example introducing a new index to support slow queries. Often, performance optimisations are discussed in the context of (de-)normalisation. While normalisation can have positive effects on database performance by avoiding redundancies and thus reducing the amount of data to be stored and loaded [9], it can introduce the need for expensive join operations which can be avoided through denormalisation [10,11]. Sanders and Shin [11] survey denormalisation strategies and propose guidelines and a methodology for choosing and applying denormalisation to improve database performance.

In contrast to relational databases, object databases are inherently denormalised since the association construct inherent to object databases supports $1:n$ and $n:n$ relationships and violates the first normal form which requires atomic attributes. This has been formalised by [12] into an algebra for the non-first normal form called NF^2 , also known as N1NF.

Database profiling is typically associated with the physical layer where cache hits or page loads might be measured. Even in an ODBMS, navigational data access is rarely monitored. One notable exception is Objectivity [13] which provides a profiling framework that recommends refactorings based on database profiling in the form of propositions for the selection of appropriate built-in persistent data types. Inspired by this approach, we took the idea further to generally optimise conceptual models based on database profiling information. In object databases, the models at the conceptual, logical and even physical levels are virtually equivalent which means that a single model has to fulfil requirements at all levels. Consequently, database performance can also be seen as a quality aspect of the conceptual model. Our experience with industry projects [2], as well as the case study presented in this paper, indicates that performance-related model optimisations may also result in a better understanding of the domain and ultimately an improved conceptual model. Unfortunately, we are not aware of research that underpins this observation.

Researchers have come up with different approaches for assessing the quality of conceptual models. Moody [14] presents a survey and classification of model quality approaches towards a common modelling quality standard. Following their terminology, our approach could be classified as *Observation-based (inductive)* in that database performance problems can point to, or even be seen as, modelling errors. They also state that research on conceptual model quality is dominated by static model analysis and there is little research on empirical model quality assessments such as profiling. Even the work presented in [15], which addresses performance optimisations on the conceptual model, relies purely on static model analysis and cost-benefit estimates. They propose conceptual model refactorings in the form of equivalence refactorings, which exclude replication-based refactorings, e.g. the duplication of attributes. In contrast, our approach is based on profiling the database in use and we make use of refactoring patterns to optimise performance.

From the human perspective, model quality can also be evaluated in several dimensions of perceived quality. A recent example is [16], which proposes several ‘soft’ metrics for the four central dimensions *perceived semantic quality*, *perceived ease of understanding*, *perceived usefulness* and *user satisfaction*.

Another approach to validating conceptual models is the use of Armstrong samples [17]. These are small datasets that adhere to all known constraints and violate as many others as possible. Such small datasets can then, for example, be discussed with end-users in order to validate the conceptual design. More recently, this concept has also been applied to analysing queries instead of data [18]. However, while such data and query analysis can be powerful and provide useful insights, it requires creation of, and is limited to, synthetic data and queries. Our concept of profiling real databases provides knowledge based on real-world data and real-world queries, including modifications to data and sequences of queries. Furthermore, database profiling comes almost for free and requires very little additional effort by users or developers.

Our approach was also inspired by the work presented in [19], where they formalised refactoring patterns and quality aspects to algorithmically analyse a model and propose refactorings according to given quality requirements. While they use only static model analysis, we applied a similar approach to the analysis of models based on real-world data. We made use of a number of refactoring patterns proposed in [20] and [7], identifying patterns that could be used to optimise a conceptual model with respect to navigational access, such as *Extract Class*, *Inline Class* and *Remove Middle Man*. We complemented these patterns with additional ones that proved to be effective in improving performance.

The refactorings resulting from our analysis are currently ranked according to expected performance improvements. Their semantic validity and feasibility have to be assessed by the model designer. One approach to supporting model designers in their refactoring decisions, at least regarding their practicality and impact on the system, has been proposed by [21]. They developed a framework that manages ‘ecosystems’ of databases and applications. The framework supports defining policies that let participants of the ecosystem respond to evolutionary events. Such a framework could for example stop a model designer from performing a refactoring that would remove an attribute from the database, even though the attribute is still required by a, currently inactive, part of the ecosystem.

In summary, existing conceptual model quality assessments usually rely either on human perception or on static model analysis. Database profiling is used only for optimisations on the logical or physical layers. This is no surprise considering the prevalence of relational databases. However, considering the suitability of object databases to agile methods, we take advantage of the equivalence of logical and conceptual models to use profiling for optimisation of the conceptual model. The contributions of this paper are the

following. We introduce a mechanism to profile navigation paths from which we draw recommendations for conceptual model optimisations, including (navigational) refactoring recommendations. Second, we present model refactorings that can be derived by analysing database profiling data. Finally, we extend the agile development cycle with direct feedback from the database.

3. Architecture overview

To validate our ideas, we implemented a system consisting of two main components: ZooDB and AgileIS. ZooDB is an open-source ODBMS ZooDB (see [22,23,4] and Section 5.3) which we enhanced with advanced profiling capabilities. AgileIS is a framework for the model-driven development of information systems which supports the design and refactoring of data models, performing static model analysis, and visualising profiling results from ZooDB. The framework consists of a graphical model editor, a model repository for storing and versioning data models, and a code generator for database application code and database evolution code (see [24,6,25,4] and Section 5.4).

Fig. 1 illustrates how AgileIS and ZooDB work together and where our two main approaches for model optimisation, *static model analysis* and *database profiling*, are implemented.

Static model analysis is part of the AgileIS framework, as shown in the left of Fig. 1. Static model analysis operates directly on the model repository and displays the results in the model editor. Compared to database profiling, static model analysis is simple, fast and does not require a database or an application to be executed. However, the results are limited to identifying unusual static model characteristics, such as classes with dozens or hundreds of attributes. The findings of the static model analysis component are presented to the model designer who can review the findings and perform corresponding refactorings on the data model.

The right of Fig. 1 shows the database profiling approach. When an application is executed by a user, as shown in the top middle of the diagram, it performs queries, navigation and data updates on a database. As shown on the right, these database operations can be recorded by a profiler in the DBMS in the form of *data access paths* which, after analysis, can be turned into recommendations for application data model refactorings that could improve database performance. The profiling information, refactoring recommendations and general data statistics can be exported from the DBMS and imported into AgileIS. The imported results are visualised in the AgileIS model editor and the model designer can review the information and confirm or reject semi-automatic execution of suggested refactorings.

Static analysis is discussed in detail in Section 4 and database profiling is discussed in Section 5.

4. Static model analysis and evolvability

Static model analysis in AgileIS focuses on model characteristics that affect two model quality aspects: *model evolvability* and *code quality*. We propose the concept of *model evolvability* as a model quality that indicates how easily a data model can be refactored, i.e. evolved. One aspect of evolvability includes how difficult the implementation of data evolution algorithm tends to be for a given refactoring. For example, evolvability tends to decrease with the increased use of inheritance in the model. Another aspect of evolvability concerns the materialised data model, i.e. the data evolution performance when a model refactoring is applied to the data in a database. For example, a small database generally tends to have a good evolvability because evolution time may be negligible. Both aspects are related to the design of the data model. However, it is equally important to consider the characteristics of the data in the database to evaluate the type and frequency of actually required refactorings. Finally, evolvability is also affected by the database evolution features of the DBMS.

In terms of code quality, AgileIS checks the data model for unusual characteristics, such as a large number of attributes in a class. For code quality metrics we mostly rely on traditional metrics as discussed in [14] or [15]. Please note that the set of metrics and

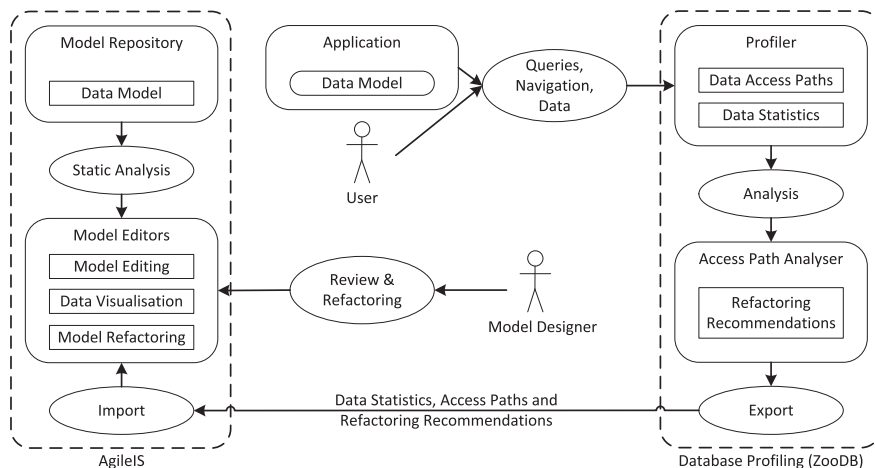


Fig. 1. Static model analysis and database profiling in AgileIS.

patterns discussed below is by no means complete but intentionally limited to those that we considered most useful for our approach. Also note that we intentionally do not give numbers for thresholds to distinguish 'good' from 'bad' classes because model quality depends a lot on the circumstances, even more so in the context of evolvability.

As we will see, code quality and model evolvability result in similar observed characteristics and suggested solutions. Also, as we discuss later in Section 6, the suggested refactorings tend to coincide with semantically sensible refactorings. AgileIS currently measures model metrics such as attribute count in classes, reference hot-spots (outgoing and incoming reference count) and the width and depth of class hierarchies.

We start with the *large classes* pattern which describes classes with more than a predefined number of attributes. This is traditionally considered to be a problem with code quality and model quality because it suggests a lack of abstraction in the code and affects maintainability. At the same time, a high number of attributes results in large objects, which can affect application performance unless the majority of attributes are accessed every time an object is loaded. This also affects database evolution performance in two ways. First, since such objects are large, evolution of related objects in a database can be costly. Second, a large number of attributes also increases the likelihood that one of these attributes will be refactored. This means such objects are not only more costly to evolve, but also more likely to be subject to evolution.

Contrary to large classes, the analyser also reports *small classes* with very few or just a single attribute. Such classes may be justified if the type of the only attribute has a variable length, such as a `byte []` or `String`. In fact such classes will be recommended by the profiler described in Section 5. However, if the type of the attribute has a small fixed length, then such a class can affect the performance and space requirements of the ODBMS because of the overhead for storing an object in the database. This overhead consists for example of storage space for the OID and the index updates for the OID-index in the database. Accordingly, performing schema and data evolution on such objects is affected by this overhead when loading, processing and storing such objects.

Another pattern is the *hot-spot* pattern. This pattern identifies classes that are referenced by many other classes or that have references to many other classes. Such highly referenced hot-spot objects can affect performance in several ways because they are likely to frequently be part of a navigation path in the data model. For example, in concurrent applications, such objects can cause lock contention, which is especially a problem if such objects are frequently updated. With respect to evolvability, hot-spot objects can be problematic depending on how well the DBMS supports schema and data evolution. For example, the ODBMS db4o [26] does not support the assignment of object identifiers (OIDs) to objects. This is a problem if a class, such as `Person`, is split into subclasses, such as `Student` and `Staff`. In such a case, the existing `Person` objects need to be migrated to their respective subclasses. Most ODBMSs, including db4o, do not support changing the class of an object, therefore the commonly used approach is to create a new instance of the respective subclass for each existing instance of the `Person` class and afterwards delete the `Person` instances. The problem now is how to update references from other classes to `Person` instances so that they point to the new instances of the subclasses. Since references in ODBMS are usually realised with OIDs, it would be sufficient to assign the OID of the original instance, which has now been deleted, to the new instance of the subclass. Any reference to the original instance would then point automatically to the new instance. Since db4o does not allow OIDs to be assigned to objects, the best remaining solution is to iterate over all objects of any other class that possibly references one of the evolved objects. At this point, the hot-spot characteristic of an object becomes significant, because, if a large part of the database potentially references these hot-spot objects, then data evolution has to load and possibly update an equally large share of the database. Avoiding hot-spot objects can reduce the cost of this type of database evolution. Limited support for class hierarchy evolution is not uncommon, but fortunately some ODBMSs, such as Versant's JVI ODBMS [27], support at least reassignment of OIDs.

At the other end of the hot-spot metric are *isolated classes*. These are classes that are not referenced from any other class and therefore are reachable by query only. For such classes, the static analyser will recommend defining an index in order to speed up queries. Since the analysis is static, the framework cannot tell whether a corresponding index already exists or there are too few instances of that class in the database to justify the creation of an index. With respect to evolvability, isolated classes have no known negative effects.

AgileIS also recognises two metrics related to class inheritance. The first is the *inheritance depth*. A high inheritance depth can affect performance in several ways. For instance, a class with a large number of direct or indirect superclasses is likely to have a correspondingly large number of attributes which results in the problems discussed above for *large classes*. For example, if we assume that any class in the model has at least one attribute, a class with 10 direct or indirect subclasses has implicitly at least 11 attributes, possibly a lot more. Also, a large number of inherited classes can indicate that instances of that class are often accessed through a superclass. This, in turn, makes it likely to have many accesses to such instances, where only the attributes of the superclasses are accessed. The loading and storing of the potentially large number of attributes of the subclasses becomes an unnecessary overhead. In other words, while the large number of attributes can be a problem in itself, a large number of inherited classes means that many of the attributes are only rarely accessed and are thus loaded unnecessarily. With respect to evolvability, a deep inheritance hierarchy can cause problems because certain changes to class hierarchies, such as insertion or removal of superclasses, are usually not directly supported in many modern ODBMSs. Instead, the schema for the modified hierarchy is usually created and the data is migrated to the new hierarchy. The likelihood for such costly refactorings is increased when the class hierarchy is complex. One solution to such deep hierarchies can be the use of delegation where attributes of subclasses are externalised to a separate class hierarchy.

The final metric is the *inheritance width* which indicates how many *direct* subclasses a class has. A large inheritance width is less problematic than a large inheritance depth, because there is no increased likelihood of *large classes*. However, a large inheritance width may mean that, depending on the application, classes are accessed through their superclasses, which means that attributes in the subclasses are loaded and stored unnecessarily. Evolvability is only a problem if the superclass needs to be evolved in a way that is not directly supported by the ODBMS, for example by adding or removing a class from which the given superclass inherits.

Description	Class	Type
Field Publication.Abstract is of type LOB together with other attributes. Consider outsourcing this field to a separate class.	Publication	LOB
Class Publication is a possible hotspot (reference count: 6).	Publication	HotSpot
Class OrganisationStructure is isolated (neither incoming nor outgoing references). Access only possible via query --> choose index.	OrganisationStructure	Isolated
Class Collections is isolated (neither incoming nor outgoing references). Access only possible via query --> choose index.	Collections	Isolated
Class Authors has no local primitive attributes (TODO: check for inherited attributes). Use shortcuts between connecting classes.	Authors	Shortcut
Class Publications has no local primitive attributes (TODO: check for inherited attributes). Use shortcuts between connecting classes.	Publications	Shortcut
Class OrganisationStructure has low (0) attribute count. Consider merging the class (TODO: is weak entity?).	OrganisationStructure	Class size
Class Publication has high (10) attribute count. Consider splitting the class.	Publication	Class size

Fig. 2. The static analysis results view in AgileIS.

AgileIS currently does not support automatic refactoring. An example screenshot of the result view of static analysis is shown in Fig. 2. Table 1 shows a summary of the discussed metrics. A detailed description of the recognised model characteristics and their implications can be found in [23].

5. Database profiling

Using database profiling in order to improve the conceptual model is much more complex than the static analysis approach discussed in the previous section. Database profiling builds on static analysis of the data model, which is available in the database in the form of the database schema, but combines it with numerous other metrics derived from database access records, data statistics and low-level database statistics such as disk-IO.

Our approach extends the agile development cycle by complementing user feedback with recommendations for model optimisation based on database profiling. The database profiler monitors navigational data access and updates, detects inefficient and performance-critical navigational access patterns and creates a set of model refactoring suggestions ranked by an estimated cost-benefit ratio.

The extended agile development cycle is shown on the left of Fig. 3 and the model optimisation workflow on the right. The cycle is extended for the test and deployment phase, where we additionally profile database use by tracking navigational data access paths. For every accessed object, we track the accessed attributes, access mode, and object predecessors in a particular navigation path. The gathered profiling information is analysed with respect to recurring data access patterns. Each detected access pattern is evaluated based on a pattern-specific cost-benefit model and, based on this evaluation, may be transformed into recommendations for model optimisations. The recommendations are concrete model optimisations in the form of model refactorings presented to the user in a graphical and textual form.

Fig. 4 shows an example of a model refactoring recommendation. Assume that a publication management system manages publications using the `Publication` class shown on the left. A publication defines a number of attributes, such as a `title` and the `text` as a string of characters. Database profiling revealed that, in 90% of cases where a publication object was accessed, only the publication title was read. However, each time a publication object was loaded, the usually large text attribute was also loaded which makes the loading of the publication expensive. As a consequence, our profiling framework would propose an *Extract Class* refactoring, illustrated in the right of Fig. 4. The refactoring suggests splitting the `Publication` class into two classes, the `Publication` class and the `PublicationText` class with a single attribute `text`. The *Extract Class* refactoring prevents the expensive loading of the text attribute each time a publication object is accessed. As a result of this refactoring, the cases where data access is targeted at the publication text become slightly more expensive, since an additional navigation from the `Publication` object to the `PublicationText` object is required. Based on the frequency of these different access patterns, the expected cost and benefit of a refactoring can be evaluated.

Table 1
List of static model metrics supported in AgileIS.

Metric	Characteristic	Impact
Large classes	Many attributes	- Lack of abstraction - Performance - Evolvability
Small classes	Few attributes	Storage overhead
Hot-spots	Highly referenced	- Lock contention - Bad evolvability if OIDs cannot be reassigned
Isolated classes	No references	Usually require index
Inheritance depth	Many direct or indirect superclasses	- Queries on superclasses may be slow - Evolvability of large classes - Evolvability of hierarchies
Inheritance width	Many subclasses	Evolvability of hierarchies

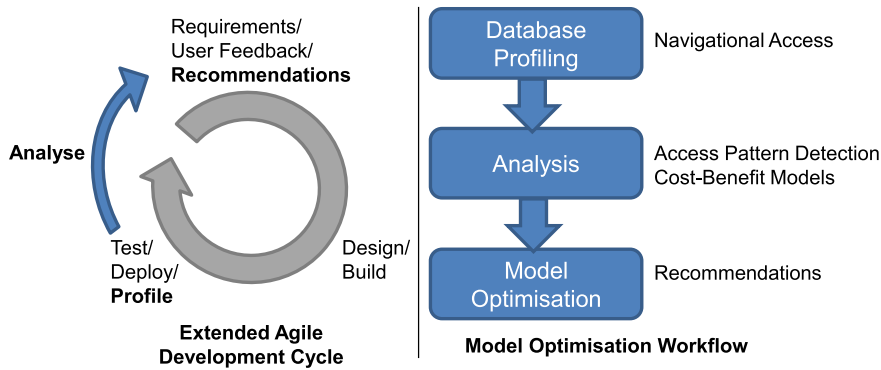


Fig. 3. Approach to model optimisation through profiling.

If the overall benefit of a refactoring is larger than the expected cost, the refactoring is recommended to the developer who can decide whether or not to perform the refactoring.

This example also illustrates that applying the results from database profiling to the conceptual application data model is facilitated by the use of ODBMSs not only through the support for navigation. When using ODBMSs, the database model is identical to the application data model so there is no need for a complex reverse-mapping of profiling results from the logical or physical database model to the conceptual application model. By this, ODBMSs again show their strength when used in agile development environments.

5.1. Tracking navigational access

For our approach, we extend database profiling so that every *navigational data access* of an application is recorded. In object databases, navigational access occurs when navigating along references between objects. These navigations form navigational access paths that can be represented as directed graphs where every node corresponds to an accessed persistent object and every edge represents a navigation along a reference. Navigational access paths typically start with a query that loads the root object of the navigation path. Once this root object is loaded, the application may use navigation to access other persistent objects. In order to obtain an application's navigational access paths, we track every access to objects and their attributes and construct a directed graph. For *object access*, we record frequency, average size of the objects per class and how the access occurred, for example as a reference from another field in another object or via a query. For *attribute access*, we track the average attribute size and distinguish read and write access.

In object databases, objects are *activated*, i.e. loaded into memory, when they are accessed and, typically, the memory is cleared after each transaction. Transactions are generally rather short to prevent locking and concurrency problems. Navigational access path profiling is transactional which means that, for each transaction, such a graph structure is created and stored upon commit(). While the resulting graphs may be similar, recording them separately for each object allows fine-grained analysis on the object level. Note that every object can be part of several navigation graphs with different shapes, for example when it is referenced from different objects.

Fig. 5 shows an example of a code snippet illustrating navigational access and the corresponding access path graph. In this example, *Publications* are associated with *UserRating* objects that represent users' ratings of a publication. In the code snippet, the average rating of a retrieved publication is computed. In step (1), a query, for example on the *title*, is performed to retrieve the publication object resulting in $n_{read} = 1$ read operations. In step (2), the user ratings are retrieved. The publication's serialised list of references to the ratings has already been loaded as part of the publication object, so accessing the list of user ratings corresponds to only an attribute access operation which means $n_{read} = 1$ attribute reads. We assume that the retrieved publication object has 345 associated user ratings. To calculate the average rating for the given publication, we iterate over the list of user ratings, as shown in (3). This results in $n_{read} = 345$ read operations to load the user rating objects. For each loaded *UserRating* object, as shown in (4), the rating attribute is accessed, which results in $n_{read} = 345$ attribute read operations. Finally, the average rating is computed.

5.2. Access path analysis

The goal of access path analysis is to detect recurrent, inefficient access patterns that may be improved through database refactoring. We rely on a set of well-known refactoring patterns for code [20] and databases [7] that we analysed for their suitability

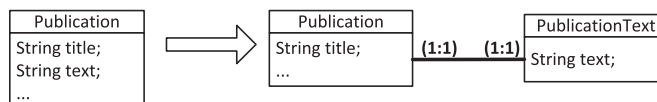


Fig. 4. Extract Class: refactoring example that extracts an attribute into a new class.

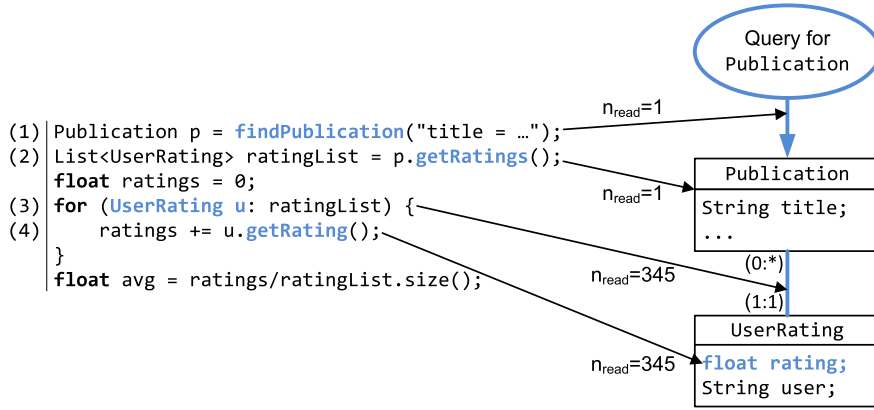


Fig. 5. Example of navigational access path.

for optimising a conceptual model with respect to database performance. We first give an overview of the supported database refactorings before presenting how we calculate the cost–benefit ratio based on which refactorings are proposed.

Figs. 6, 7 and 8 show a selection of refactoring patterns which have been integrated into our profiling framework. The original model is shown on the left of each pattern and the optimised model on the right. The **Store Aggregate** refactoring in Fig. 6 is the pattern that would have been applied to the aggregated rating example discussed in Fig. 5. It is generally applied in cases where expensive computations of aggregates are repeatedly computed. The pattern suggests an additional attribute, in our example `Publication.avgRating`, to be introduced where the computed aggregation is stored. Of course, aggregated values have to be updated and it is the responsibility of the developer to decide whether such a refactoring makes sense.

The pattern **Extract Class** and its inverse **Inline Class** are shown in Fig. 7. **Extract Class** extracts one or more attributes of a class into a new class and may be based on attribute access and attribute size, as in the example presented in Fig. 4 where the publication text was extracted from the publication class since it was large and rarely accessed. Also, this pattern may be applied if the analyser identifies two or more distinct sets of attributes that are never accessed in combination. In contrast, the **Inline Class** is applied to merge two classes, for instance if they have a 1:1 relation and attributes from both classes are typically accessed together.

The **Remove Middle Man** pattern, shown in Fig. 8, recommends the insertion of a direct reference between two classes to avoid navigation over intermediate objects. In the example, a direct association was created to access the author of the leading (first) keynote for each conference, as this was a recurrent access pattern in the application.

In addition to the patterns shown above, we support other patterns such as the insertion of duplicate attributes, or removal of unused attributes.

In order to select and propose a refactoring pattern in the analysis phase, we calculate the cost–benefit ratio for each possible pattern. $r_{b/c} = \frac{\text{benefit}}{\text{cost}}$ is based on estimates for *cost* and *benefit* which are measured solely in terms of bytes that need to be read or written when executing a certain access pattern. This simplified approach allows us to calculate cost–benefit estimates that are almost independent of hardware, disk-layout, database size or workload on the database. Instead, we require only a small representative dataset to suggest useful refactorings. The benefit here is that profiling can be performed by developers with reasonable accuracy, even across software releases, without requiring access to the full databases and hardware of end-users.

We calculate the cost–benefit ratio $r_{b/c}$ for each pattern as follows:

$$r_{b/c} = \frac{\text{benefit}}{\text{cost}} = \frac{\alpha_r * \sum n_r * \text{size}_{r,\text{avoided}} + \alpha_w * \sum n_w * \text{size}_{w,\text{avoided}}}{\alpha_r * \sum n_r * \text{size}_{r,\text{added}} + \alpha_w * \sum n_w * \text{size}_{w,\text{added}}}$$

The subscripts *r* and *w* indicate read and write terms, for instance α_r and α_w are weighting factors for the different costs of reading and writing data. The benefit is the sum of all avoided operations that read or write bytes due to the refactoring. The cost is the sum of all operations that read or write additional bytes. Inside each sum, we multiply the number of occurrences n_r or n_w of a refactoring by

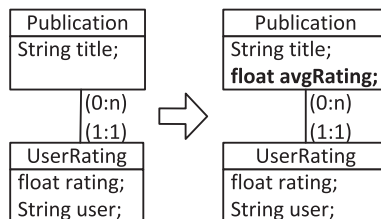


Fig. 6. The store aggregate refactoring pattern.

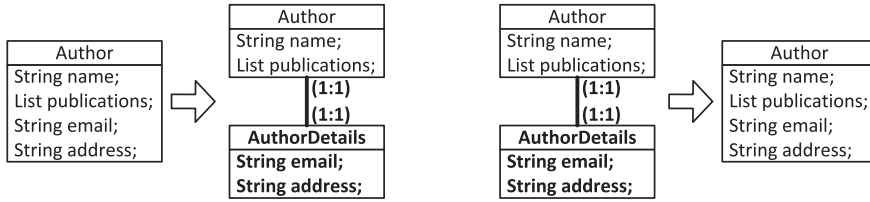


Fig. 7. The Extract Class (left) and Inline Class (right) refactoring patterns.

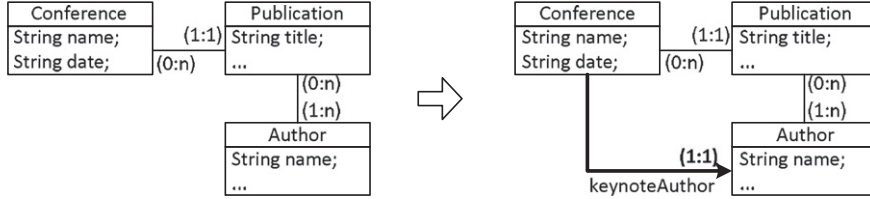


Fig. 8. The Remove Middle Man refactoring pattern.

the average number of bytes for each operation. For n and $size$, we use the access frequency and average sizes of objects and attributes that we recorded in the access graphs described in Section 5.1.

As an example, consider the *store aggregate* refactoring from Fig. 6 to the access path example in Fig. 5. In the refactored model, the loading of the *UserRating* objects is avoided by storing the average rating directly in the *Publication*, which corresponds to the *benefit* of the refactoring. The *cost* of the refactoring is represented by the aggregated costs that originate from the fact that the publication objects have an additional attribute, and are thus larger in size. The cost also includes cases that read *Publication* objects but do not require the *avgRating* attribute, and cases that require *avgRating* but also access the *UserRating* since they need access to the user name. Also, it includes cases that create or update *Publication* instances and cases that update *UserRating.rating*, which now have to update *Publication.avgRating* as well. For all these cases, we need to sum up the costs in terms of bytes to read or write in order to calculate the correct ratio $r_{b/c}$.

Profiling may show that a *UserRating* is 25 bytes and the *avgRating* attribute has four bytes. For simplification, we assume that the example in Fig. 5 represents the only case where publications and user ratings are accessed and that we access 15 *Publications* with 5 ratings each. Since α_r can be reduced, cost and benefit would be calculated as follows:

$$r_{b/c} = \frac{\text{benefit}}{\text{cost}} = \frac{\alpha_r * n_{r,\text{UserRating}} * \text{size}_{\text{UserRating}}}{\alpha_r * n_{r,\text{Publication}} * \text{size}_{\text{avgRating}}} = \frac{(15 * 5) * 25\text{bytes}}{15 * 4\text{bytes}} = 31.25$$

The resulting ratio $r_{c/b} = 31.25 \geq 1$ indicates that the refactoring should be suggested to the developer since the benefit outweighs the cost. Note that $r_{c/b} = 31.25 \geq 1$ does *not* indicate that the refactored database may be 30 times faster. The ratio only indicates that the expected *gain* of the refactoring is bigger than the *cost* of the refactoring, which means that a gain of factor 30 is expected only for the part of the database access that involves the refactored part of the data model. For instance, in the example used above, the calculation does not consider the performance of the query.

We currently do not calculate the cost–benefit ratio for combinations of refactoring patterns. If not done carefully, estimating the cost of combinations of refactoring patterns can require exponential computational complexity if each possible combination is calculated separately. Finding an efficient algorithm for simulating combinations of refactoring patterns is therefore subject to further research.

5.3. Integration with ZooDB

The profiling capabilities for navigational access paths were implemented in the open-source system ZooDB, see [22]. ZooDB is a pure Java object database that provides partial support for the JDO 3.0 standard.²

All application classes to be made persistent need to be implemented as subclasses of the ZooDB class *PersistentCapableImpl*. Methods that access class fields are enhanced with method calls to either `activateForRead("attrName")` or

² <http://db.apache.org/jdo>.

activateForWrite("attrName"), as shown in Listing 1. While currently not implemented in ZooDB, the JDO standard suggests that the code for these enhancements be injected automatically at compile time, thus relieving the developer of doing this manually.

Listing 1. Simplified example for automatic insertion of database callbacks.

```
public class Author extends PersistentCapableImpl {
    private String name;
    public String getName() {
        activateForRead("name");
        return name;
    }
}
```

The data structure for storing navigational access paths is shown in Fig. 9. In the bottom, it shows persistent application classes, such as *Author* or *Publication*, that extend *PersistentCapableImpl*.

Every time a persistent object is activated, we record the activation as an instance of *Activation*, along with a reference to the transaction and how the activation occurred, for example as a reference from a field in another object (*rootField*) or via a query. If a second object is activated via navigation from the first, a new instance of *Activation* is created and registered as a 'child' of the first *Activation* instance. Every time a field is accessed, the field access table of the corresponding *Activation* object is updated with field access details, such as number of reads and writes. If an accessed object has already been activated, the *Activation* instance is reused and the field access counters incremented accordingly. The average sizes of objects and attributes are stored separately from these graphs in the central *ClassSizeRepository* which records class size statistics independently for each persistent class in instances of *ClassSizeStats*.

Fig. 10 gives an overview of the architecture of ZooDB with a focus on components that were affected by the implementation of the database profiling framework. The *transaction API* and the *query API* contain callbacks for general profiling management, such as associating navigation graphs with transaction contexts or assigning queries as roots of navigation paths. The *object read* and *object write* components serialise and de-serialise objects to and from byte streams. These components were enhanced to record the number of bytes that are read or written for each object. The data collected by these components is stored in the *graph archive*, which maintains all graphs created while profiling. The *graph analyser* is executed after an application run to analyse the gathered graphs and compile recommendations for the developer. The disk-IO component, in the lower part of the figure, records the number of page reads and writes. While the number of pages is not directly used in the calculation of refactoring recommendations, it is recorded and, as shown in Section 5.5, can be used to compare the number of page-IOs before and after a refactoring to provide a hardware-independent way of estimating the usefulness of a refactoring.

After the graph analyser has processed the gathered profiling information, the resulting recommendations can be exported as an XML file. Class statistics, recorded queries and the data access paths are exported to a separate XML file.

5.4. Integration with AgileIS

The profiling capabilities in ZooDB provide recommendations that can be integrated into application data models using AgileIS. To achieve this, AgileIS has been extended so that profiling information and recommendations from our profiling framework exported as XML can be imported into AgileIS and presented to the model designer through the graphical model editor. Fig. 11 shows a screenshot of the recommendation preview screen of the AgileIS environment. The imported recommendations are shown textually in the lower part of the screenshot.

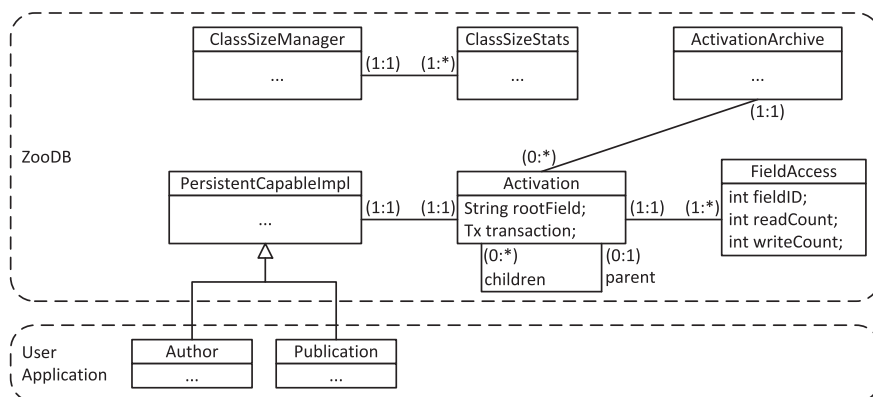


Fig. 9. UML structure for tracking object access graphs.

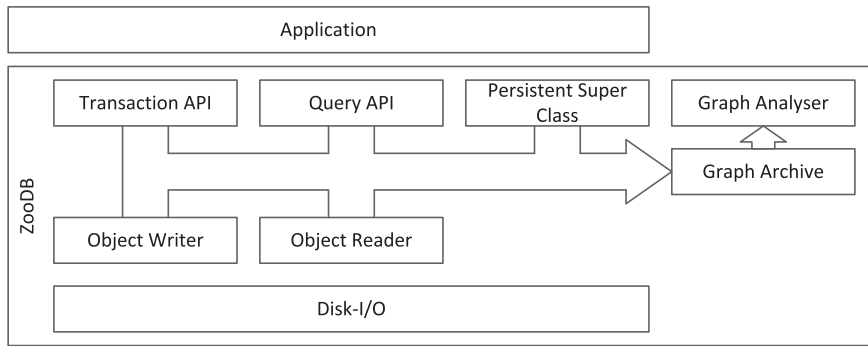


Fig. 10. ZooDB architecture overview.

As shown in the magnified refactoring view in Fig. 12, each recommendation consists of a description that details the proposed refactoring. The descriptions can be unfolded, as demonstrated for the *Large Object* refactoring on the *Publication* class, to show how often an access path was recorded, what the average size of the accessed data was and what the expected cost and gain of the proposed refactoring is. The other columns show the name of the affected class, the name of the refactoring pattern and the gain/cost ratio. The horizontal slider above the recommendation list allows a threshold for $r_{b/c}$ to be set below which recommendations are filtered out.

In the model view in the upper half of Fig. 11, a preview of the selected recommendation is highlighted in green and red, in this case for the outsourcing of the *Publication.abstract* attribute into a separate class. The green box in the top left corner represents the new separate class. The attribute that is framed in red in the top of the *Publication* class represents the attribute that is outsourced and thus removed from the class. This is the refactoring depicted in Fig. 4. Besides previewing recommended changes, AgileIS also supports automated application of a recommended refactoring to the current model. While such refactorings can typically be performed automatically, some refactorings require additional work by the developer. For example, for the *store aggregate* refactoring, which automatically creates an aggregate attribute, the name and type of the aggregate attribute may need to be adapted by the developer. The application logic for calculating or using the aggregate is currently not generated by AgileIS.

As shown in Fig. 13, AgileIS can also visualise the raw profiling data. The screenshot shows on the left a list of all persistent classes including how often they were accessed or written. If a class has been accessed in a recorded application run, then the class can be

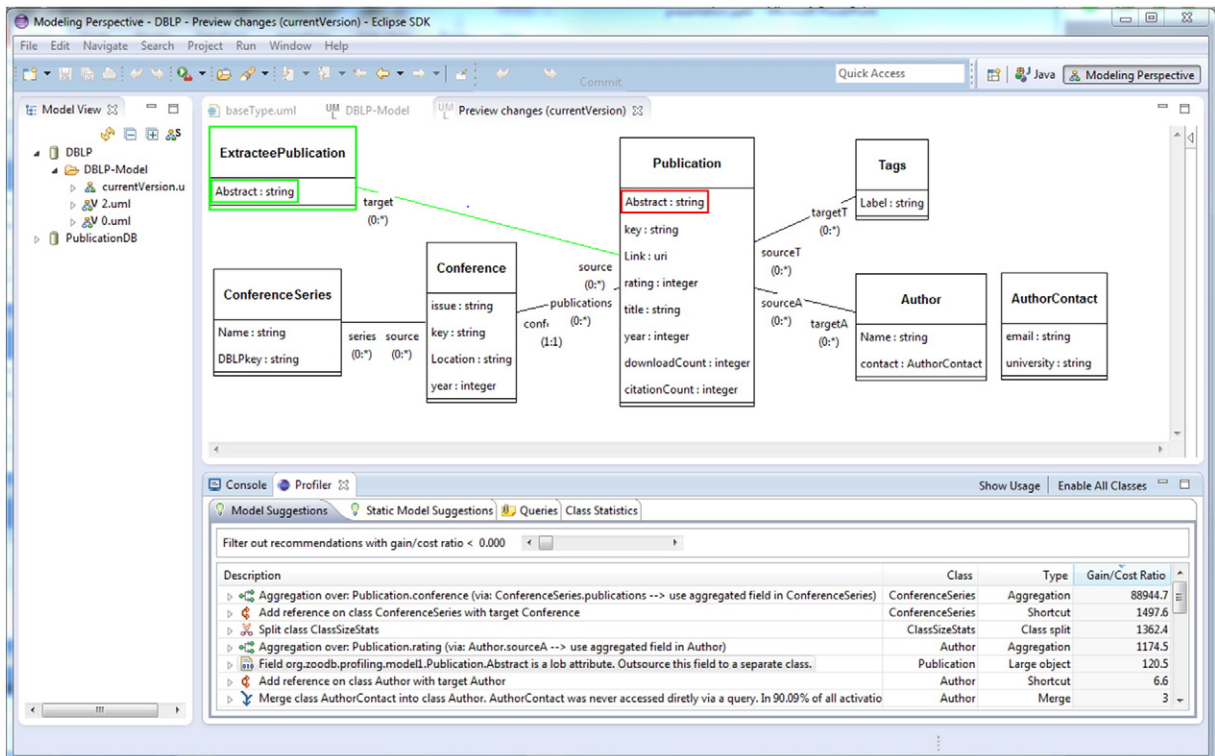


Fig. 11. The AgileIS model editor showing a suggested refactoring preview.

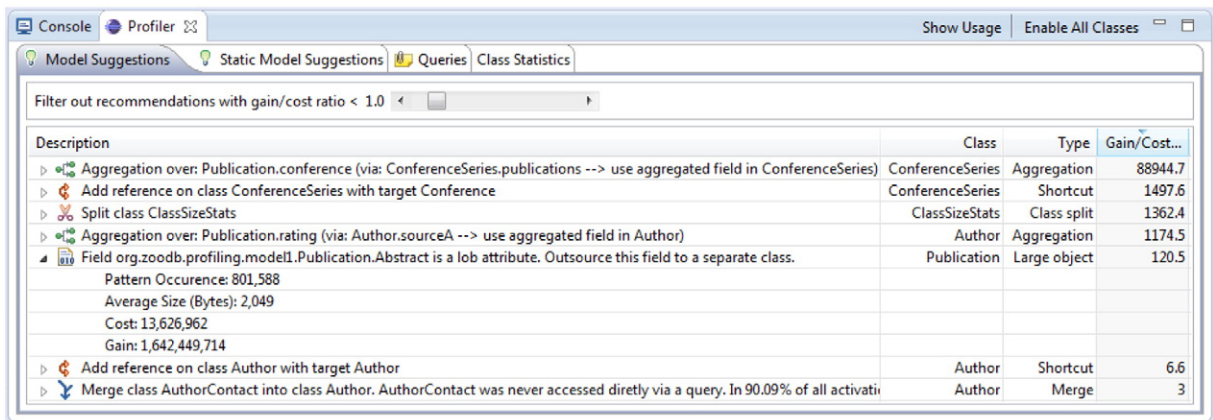


Fig. 12. The AgileIS refactoring recommendation view.

unfolded, as shown for the `Publication` class, and the access paths are shown. For the `Publication` class, three access paths are available. The first, which was called 277,580 times, was a navigation via the `Author.publications` attribute. The second was access through a query and the third access via navigation from `ConferenceSeries.publications`.

These statistics are also visualised on the right of the screenshot in a pie chart. The pie chart shows that `Publication` objects were accessed through three distinct *access groups* which are shown in the bottom of the view. The pattern occurrence count indicates that the access groups (1) and (2) belong to the access path coming from `Author.publications`. This means that each navigation from a related `Author` resulted either in access to the `title` and `year` attribute or to the `rating` attribute, but never to both groups of attributes or any other attributes. Access groups (3) and (4) can be directly matched to query access (199,999 occurrences) and navigation from `ConferenceSeries` (323,996 occurrences).

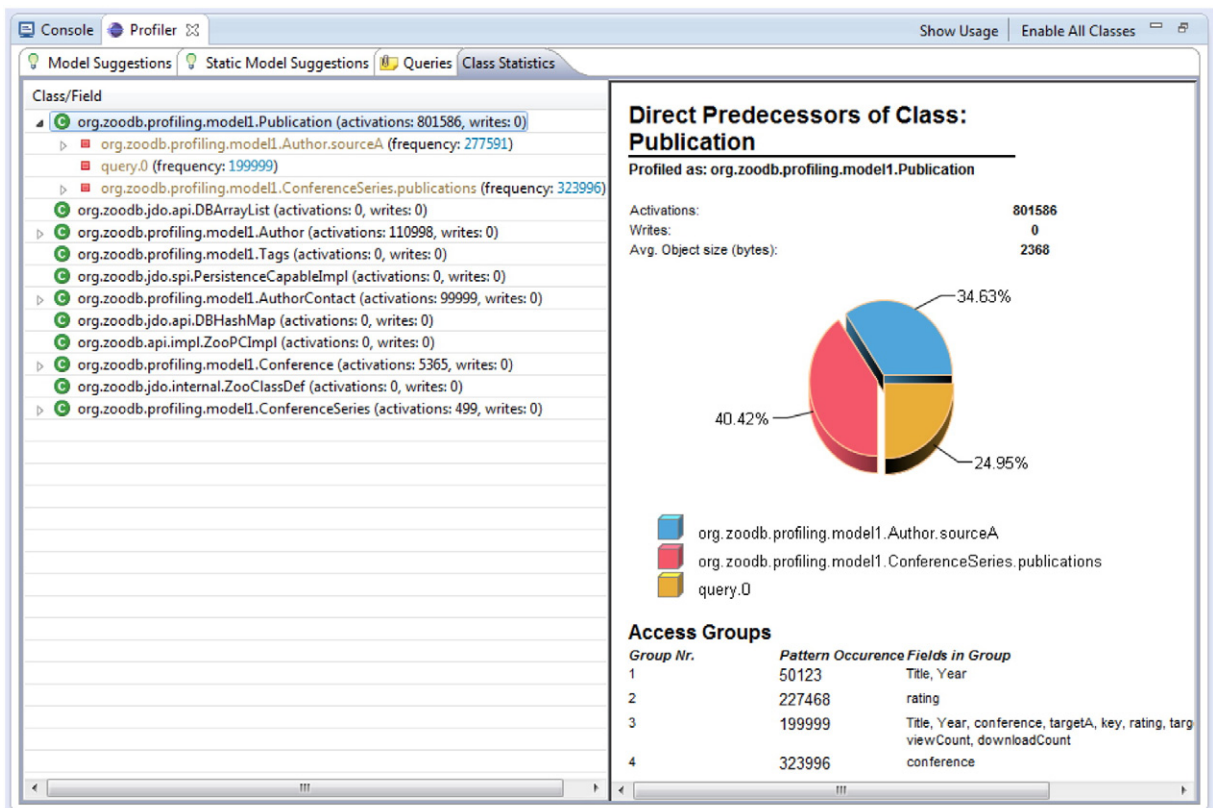


Fig. 13. The AgileIS profiling statistics view.

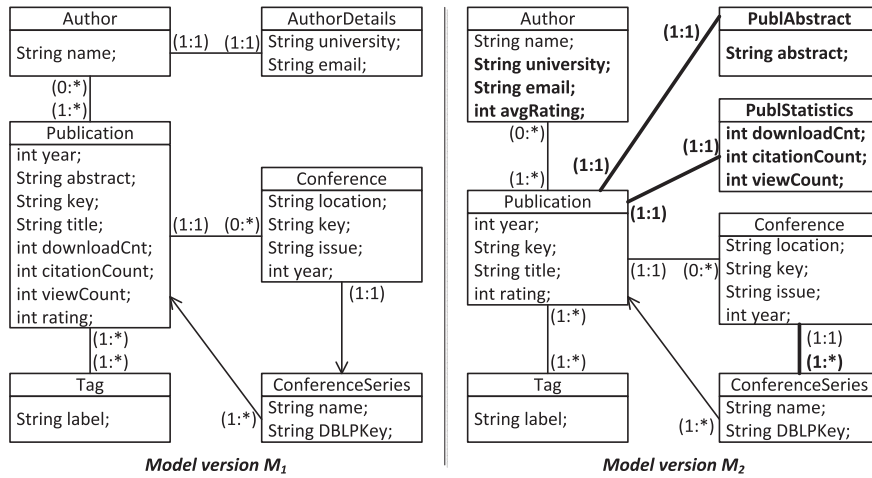


Fig. 14. Data models before (M_1)/after optimisation (M_2) (changes in bold).

5.5. Experimental validation

To validate the approach, we implemented an information system for managing scientific publications. Then we compared the performance of the two versions of the persistent data model shown in Fig. 14. Model M_1 on the left is the original model, for which we performed database profiling. It should be noted that we deliberately introduced a severe modelling mistake in M_1 , where the class *ConferenceSeries* does not have a direct relation to *Conference* but only to *Publication*. Model M_2 on the right of Fig. 14 is an optimised version of M_1 , achieved by applying model recommendations proposed by our profiling framework based on the profiling data of M_1 . As test data, we used a snapshot (March 2013) of the DBLP scientific publication archive [5], which we extended with publication abstracts that were generated as random sequences of 1000 characters. The size of the original dataset was 1.1 GB. Table 2 lists the numbers of specific objects this dataset contained.

Table 3 shows the five different use-cases that we used in the performance tests. Each use-case consists of a query followed by navigation operations.

The use-cases performed mostly read-only operations and were each run at least 500 times. Columns M_1 and M_2 in Table 3 show the performance results from executing the use-cases on a database with the respective model version. When running the use-cases on M_1 , the analyser provided a set of 28 refactoring recommendations, from which we chose 6 refactorings from the top 7 as shown in Table 4, ranked by $r_{b/c}$.

In Fig. 14 M_2 , the applied refactorings are highlighted in bold. According to the recommendations, an aggregate attribute *Author.avgRating* was added to the *Author* class and the class *AuthorDetails* was merged with the *Author* class. Furthermore, the *Publication* class was split into two classes *Publication* and *PubStatistics* and the abstract attribute was extracted into a separate class *PubAbstract*. As noted before, we deliberately introduced a severe modelling mistake in M_1 , where the class *ConferenceSeries* does not have a direct relation to *Conference* but only to *Publication*, as shown in the left of Fig. 15. This mistake affects use-case UC_1 which retrieves conferences from a set of conference series and, as a consequence of the current model, has to iterate over the associated *Publication* objects in order to retrieve the conferences.

The results show that the suggested refactorings clearly improved database performance in terms of page reads. The biggest improvement is seen for use-case UC_2 optimisations with a more than 100-fold improvement. The test loaded 10,000 *Author* instances from the databases and calculated the average rating for each one. For M_1 , this involved loading all associated *Publications*, while, for M_2 , only the authors had to be loaded.

The second biggest improvement is seen for use-case UC_1 , where the recommendations R_1 and R_2 from the profiling analyser fixed the modelling mistake. R_1 and R_2 are essentially equivalent and resulted in a single refactoring which created a (1:*) relation from *ConferenceSeries* to *Conference*. The high $r_{b/c}$ of R_1 is a bit off since the analyser did not correctly understand the semantics of what was done. The analyser only saw that UC_1 was iterating through all *Publications* of a conference and optimistically assumed

Table 2
Statistics of imported DBLP data.

Class	Number of instances
Authors and AuthorDetails	821,779
Publications (inproceedings)	1,240,494
Conferences	19,899
ConferenceSeries	3268
Tags	55,978

Table 3Use-cases with related refactorings and average page loads in M_1 and M_2 .

UC_{ID}	Use-case	Related	Page loads	
		Refactorings	M_1	M_2
1	Get Conferences for ConferenceSeries	1, 2, 5	246,577	2867
2	Get average rating of Publications per Author	4, 5	155,040	1498
3	Get Publication title and year	5	35,893	13,535
4	Access Author and AuthorDetails together	7	17,967	10,341
5	Access statistics data in Publication separately	3, 5	58,636	9343

that the result of this iteration could be aggregated into a single value. In reality, however, the result of the aggregation was not a single value but a set of all associated `ConferenceIssue` instances. In summary, the analyser effectively detected a semantic modelling error by means of profiling a semantically correct use-case. This correlation of performance problems with semantic problems is discussed in more detail in Section 6.

Other than that, while the estimated cost–benefit ratios $r_{b/c}$ did not match the test results perfectly due to interaction effects, they gave a very good indication of the usefulness of a refactoring.

5.6. Conceptual model refactoring

Refactoring the conceptual model based on performance measurements of course raises the concern that the refactoring may affect the perceived quality of the conceptual model. However, if we look at the model shown in Fig. 14, we can see that the refactored version does not look ‘bad’, in fact, at least regarding the modelling mistake with the `ConferenceSeries`, it looks better than the original version.

We believe that the conceptual quality of the refactorings is not coincidental but can, at least in part, be explained by our approach to profiling. One reason is that the recorded access paths cannot cross transaction boundaries. Since transactions often encapsulate operations related to a single task executed by a user, they indicate a semantic context. On the level of individual access paths, the argument is similar. Access paths always start with a query followed by a sequence of navigation operations. Therefore, just as the transaction as a whole, an access path is likely to represent a single task or even a part of a task executed by a user which gives an even stronger indication that all operations in the access path have some common semantic context.

If we now look at the refactorings, such as **Inline Class**, **Store Aggregate**, or **Remove Middle Man**, we see that they move elements of the model closer together if these elements frequently occur in the same access path. If occurrence in the same access path indicates semantic proximity, then it is no surprise that moving these semantically related elements closer makes sense on a semantic level, which means it is likely to look sensible in a conceptual model. Accordingly, elements that are rarely accessed together are likely to have a weaker semantic relationship and can therefore be moved apart by **Extract Class** without impacting the perceived quality of the conceptual model.

For the modelling mistake in our example in Fig. 15, the connection is more obvious. In order to extract the `Conference` instances for a given `ConferenceSeries`, the developer implemented a ‘workaround’ by traversing the `Publication` instances of a `ConferenceSeries`. Almost by definition, a ‘workaround’ implies additional effort to solve a task. It is hence no surprise that the workaround shows up in the profiling data as unexpected workload. Profiling can thus be seen as a way of detecting workarounds which in turn indicate a mismatch between the problem domain and the implementation of the application.

Concerning the growing complexity of the data model due to the new reference from `Conference – Series` to `Conference`, it is interesting to note that when we profiled the refactored model M_2 , one resulting suggestion was to remove the reference from `ConferenceSeries` to `Publication` because it was not used any more. This means that the additional complexity introduced through R_1 and R_2 was amortised by the removal of another relationship in a follow-up profiling session. The total model complexity did in this case not increase.

6. Semantic aspects

The main advantage of database profiling is that it can provide real-world database usage and data statistics. While this allows quite accurate generation of refactoring suggestions for performance improvements, it sometimes goes far beyond that by indicating

Table 4Recommended refactorings and their cost–benefit ratio $r_{b/c}$.

R_{ID}	Refactoring recommendation	$r_{b/c}$
1	Aggregation <code>Publication.conference</code> → <code>ConferenceSeries</code>	88945
2	Remove Middle Man <code>ConferenceSeries</code> to <code>Conference</code>	1498
3	Split Class <code>Publication</code> into <code>PublicationStatistics</code>	1362
4	Aggregation <code>Publication.rating</code> → <code>Author.avgRating</code>	1175
5	LOB Attribute <code>Publication.Abstract</code> into <code>PublAbstract</code>	120.5
7	Merging Class <code>AuthorDetails</code> into <code>Author</code>	3.0

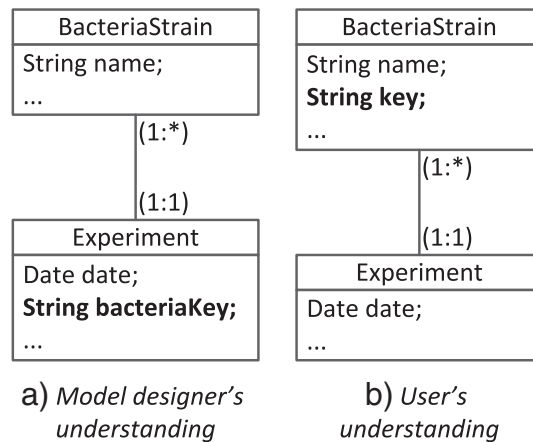


Fig. 17. Semantic understanding of the problem domain by the model designer (a) and the user (b).

ConferenceSeries should reference all Publications that were ever published within the conference series, as shown in the left of Fig. 15. In a way this is *not completely incorrect*, because all of the required information is stored in the model, and the model does not contain any associations that are actually incorrect. While the case study is based on artificial modelling and modelling mistakes, it is not hard to imagine that similar errors may occur in real projects with more complex domains.

If we had applied standard database model normalisation to the database schema, the modelling mistake may not have resulted in performance problems. However, while normalisation would have cured the *symptoms* in terms of bad performance, it would not have cured the *cause* of the problem which lies in a conceptual model of poor quality. Considering this, database profiling on the conceptual model has a unique strength in providing semantically underpinned feedback on a high (conceptual) level in the development chain. It should be noted that a watchful database administrator may report refactorings of this kind to the designer of the conceptual model. However, it is questionable how common or reliable such a feedback route is, especially since a database administrator is likely to know even less about the problem domain than the model designer. An automatic feedback system for these kinds of issues would definitely be useful.

6.2. Data statistics

Data statistics are another area where semantic misconceptions could be detected by means of profiling. For instance, assume that the database of a laboratory needs to store data about experiments with bacteria. The user requirements say that the bacteria used in any experiment should be provided with a key for identification. The model designer may understand this as requiring that the bacteria involved in each experiment should be assigned a key, possibly thinking that the users want to distinguish some kind of substrains or similar. This is depicted in Fig. 17(a). What the user had in mind though, as shown in Fig. 17(b), may have been that simply each strain of bacteria has a unique key, which means there may be only one key required for each of the, say, 10 different strains of bacteria used in the possibly 10,000,000 experiments. This misconception may not necessarily result in a functional deficit, because the users can still enter the keys after each experiment and can use them to look up experiments that involve bacteria of a given strain. The problem occurs only on the data level, where the database stores 10,000,000 keys instead of just 10. This could in extreme cases also result in performance issues, however it would certainly turn up in detailed data statistics collected from a database. Also, the users may be annoyed because they have to enter the bacteria key anew for every experiment. Though, as long as no user enters a wrong key, uniqueness is still maintained as expected by the user. While the data property of uniqueness is not recorded in the current implementation of ZooDB, such a uniqueness statistic may be a worthwhile addition to the existing counting of objects and bytes.

6.3. Benefits of non-normalised models

It may be worthwhile to investigate whether performance-related refactorings derived from relational databases expose similar information when translated to the conceptual level. If they do not, then one possibility is that the semantic correctness of refactorings derived from ODBMSs is owed to the fact that, compared to the relational model, conceptual data models tend to be less normalised, where normalisation is often perceived as 'clean', but not necessarily as natural or intuitive.³ Indeed, one could see denormalised properties of a model, such as the presence of reverse references or shortcut references, as additional semantic information stored in a data model. This would mean that a human-designed conceptual data model carries more semantic information *before* normalisation is applied while it better resembles how a human being understands the world.

³ Please note that while normalisation is sometimes considered rather desirable in academia, normalised databases appear to be rare in practice [30].

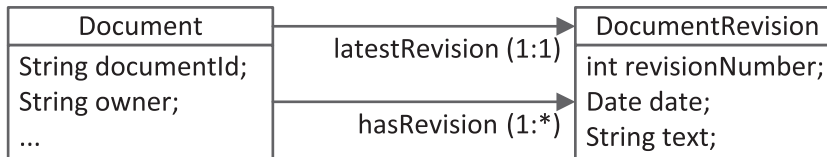


Fig. 18. An example of a data model with a semantically meaningful shortcut reference (`latestRevision`) which would be removed when mapped to a fully normalised relational model because it is implicitly defined through the attributes `hasRevision` and `revisionNumber`.

An example is shown in Fig. 18, which shows a small document versioning system with multiple document revisions. The attribute `latestRevision` simply references the latest revision in the set of revisions. However, the latest revision could also be extracted by searching for the revision with the highest `revisionNumber`. The attribute `latestRevision` is therefore redundant and would be removed when the model is mapped to a fully normalised relational model. However, since users of the system are predominantly interested in the latest version of their documents, `latestRevision` carries some important semantic information in the sense that it reflects the fact that the latest version has a special meaning to the users and is by far the most likely version to be accessed. The normalisation process effectively prunes this information from the model.

With respect to performance, the pruning of the shortcut reference means that a fully normalised relational database would have to perform a query over all document revisions that belong to a certain document joined with the highest assigned revision number for any revision of the requested document. This means that in this case, pruning the semantic information about a frequently used association results in a considerable performance cost. Of course, this cost can be reduced by the use of materialised views, which, in a way, adds the pruned semantic information through the back door.

In summary, also considering the modelling mistake presented previously in Fig. 17, we think that such examples do not accidentally represent a coincidence of a semantic problem and a performance problem. Instead, the flow of information suggests that there may be a deeper connection and that such ‘coincidences’ may occur regularly. Therefore, applying database profiling to the conceptual model should be further investigated, because it offers feedback and opportunity for corrections higher up in the development chain than the traditional profiling feedback on the level logical models. Applying such feedback only on the level of logical models would potentially waste the semantic information contained in the profiling data. Based on this, it might be useful to consider performance as an aspect of the *quality* of conceptual data models. Profiling on the level of conceptual data models should be a promising field for future research.

6.4. Model quality

There are many aspects of model quality. However, many of these aspects, even some of those that can be measured with various metrics, often give no absolute indication of whether a model is good or bad. Model quality can be subjective and lies to some extent in the eye of the beholder.

This is in part caused by the fact that the conceptual model reflects the intuitive human perception of several aspects such as the problem domain, software engineering practices, and technical and architectural constraints, including mistakes and misunderstandings. This in turn means that the perceived quality of a conceptual model is not only inherently subjective to some degree, but also driven by much more than only semantic understanding. While the awareness for these characteristics is not new, database profiling should be seen as a new approach to improving model quality on the conceptual level and has a good potential because it measures the model directly against its eventual purpose, i.e. the realisation in an information system used by end-users. This means that profiling is not so much a measure for information flowing *into* a model, such as user requirements, but a measure for how well a model behaves in face of the original real-world problem domain.

In this section, we discussed how and why model quality is related to database profiling. However, establishing this connection properly requires detailed analysis which is a topic for future research.

7. ODBMS in agile development

We now turn to consider the advantages of ODBMS compared to RDBMS in the context of agile development and database evolution. We briefly review some widely known advantages and then discuss some more subtle differences and their implications.

This research relies heavily on the use of ODBMS. While ODBMS are still not widely used, they are common in certain industries such as aviation and defence. Our decision to use an ODBMS was initially motivated by our positive experience with an ODBMS in a real-world project called HCSS [2,4]. The HCSS project is the Herschel Common Science System which supports many tasks related to the operation and scientific processing of the Herschel Space Observatory of the European Space Agency.

In the context of agile development practices, RDBMSs have the known disadvantage that they require several logically independent models, at least the logical database model, the application data model and additional views, and possibly complex mapping layers to map the data and queries between these models. For evolving models, ODBMSs have the advantage that the database model matches the application data model. This means that changes to the data model can be directly transferred to the databases and no mapping layers need to be adapted. Also, refactoring of queries is often easier because ODBMSs tend to require fewer queries

than RDBMSs due to their support for navigation which is native to object-oriented languages. These points are discussed by [3] and also match our own experience with the case study of the HCSS project.

Another often discussed topic is the *impedance mismatch* [31] between the object-oriented programming model and the relational database model. The mismatch includes class inheritance, which is for example relevant when performing queries on superclasses and n:m relations, which have to be mapped to separate relationship tables in RDBMS as well as variable length attributes, which do not fit naturally with the concept of fixed length rows. Finally there are directed 1:n relations ($A \rightarrow B$), which have to be inverted into n:1 relations and require an additional index on the target side B to efficiently identify relations that are associated with A . This inversion also makes it harder to implement ordered associations.

A general discussion of ODBMSs vs. RDBMSs is given in [32], however they hardly discuss issues related to logical independence, denormalisation, encapsulation, code proximity, or knowledge transfer between database administrators and developers in agile development.

The first point is the lack of logical independence of conceptual and logical data models when using an ODBMS. Logical independence allows independent optimisation of the two models and even helps avoid certain types of schema or data evolution. For example, the physical unit (such as meter versus inch) of stored values does not need to be the same as the physical unit used in the application domain.

However, one could argue that the need for logical independence is higher when using an RDBMS compared to an ODBMS since RDBMSs break the object-oriented paradigm of *encapsulation*, which in itself provides a form of logical independence [33], and therefore it needs to be introduced at another point. In other words, the benefits of logical independence provided by views and corresponding mapping layers are similar to the benefits of encapsulation in the form of APIs and their (separate) implementations.

One advantage of the encapsulation-paradigm compared to RDBMSs is that it promotes hierarchical call trees in code and helps to avoid code duplications by allowing code reuse. This is useful in agile development with frequent refactorings because, due to the hierarchical approach, changes are often very local or even confined to a single class. Contrary to that, SQL queries do not allow code reuse which means that any refactoring of the model requires checking and refactoring of a potentially large set of SQL queries. In an RDBMS, code reuse can be achieved with stored procedures or user-defined functions, however, using procedures and functions somewhat defeats the idea of an imperative query language and thus can be seen as a case in point for encapsulation and call trees. Also, for ODBMSs, due to the use of navigation instead of queries, a given refactoring is likely to require a smaller number of queries to be updated and the updates to queries are probably smaller and simpler. Due to encapsulation, updates to navigation code should be very localised.

A more complex point is that encapsulation affects denormalisation in ODBMSs, in particular duplication of values, derived attributes and other functional dependencies. While such denormalisations appears to be common in RDBMSs [30], they pose a problem because they are hard to maintain. Several queries may update certain fields independently which would lead to inconsistencies. This can be partly avoided by triggers or materialised views, however these would be outside the normal source code of the project and therefore easier to overlook and consequently harder to maintain during refactorings.

Surprisingly, in this context, it is a strong advantage of many modern ODBMSs that their query languages tend to support only the reading but not the writing or updating of objects. This increases encapsulation because it ensures that write access to objects can only occur through normal program code such as setter methods. This means that in an ODBMS there is only a single place in the code that needs to be updated if the duplication is refactored and this place is in code-wise proximity of the affected attributes because they are in the same class definition. Especially in agile development such duplications can be very useful. For example in the HCSS project, duplication of values was sometimes introduced on a trial basis in order to evaluate the effects of using different access paths. In the case that an experimental access path did not work as expected, the application could be easily switched back to use the old access path without requiring schema evolution. This is especially useful with real-world data which can have unpredictable characteristics [2]. In the context of database evolution, especially for data migration, it can also be useful to keep deprecated attributes or classes and remove them only in a later evolution cycle. For such deprecated structures, the concept of read-only queries makes it easy to ensure that no further write access occurs to such attributes or classes and inconsistencies can be avoided. This advantage of ODBMSs is clearly not limited to simple duplications but to all kinds of dependencies or constraints that involve multiple attributes.

Regarding the definition of such constraints, the ODBMS concept of using a single data model for all applications, rather than multiple relational views, moves the task of defining class invariants and constraints away from the database developer into the hands of the developers who implemented the persistent classes. This has the advantage that these constraints are implemented by the developers who are responsible for the application logic, thus avoiding the step of having to explain these constraints to a DBA to implement. Furthermore, constraints are directly visible in the code, which improves maintainability.

Finally, the reduced effort for evolving the data model and databases may allow these tasks to be performed by a single developer as a part-time job. For example for the HCSS project, the use of an ODBMS paid off well. Over the course of more than 5 years, the database evolution developer spent on average around 3 days per month on data evolution, based on a software release interval of 6–8 weeks. This included 35+ database evolution cycles consisting of hundreds of changes on the 250 persistent classes. For each software release, database evolution was performed on over 100 databases, some exceeding 100 GB in size.

8. Discussion

The AgileIS framework uses two approaches to model optimisation. Static model analysis is directly available during model development and can thus prevent modelling errors before they become part of a software release. However, the problem analysis and refactoring suggestions resulting from static analysis have somewhat limited accuracy, because the UML modelling language used

in AgileIS does not support rich semantic contexts for the modelled concepts. One solution to improving this would be to add other model types to AgileIS that provide modelling of additional aspects such as expected data statistics, semantic context of attributes and classes or expected (or permitted) queries and navigational access paths. Such features would improve the usefulness of static analysis in combination with its unique strength of permitting virtually immediate feedback to the model designer.

The other approach for model optimisation that is available in AgileIS is the support for database profiling. AgileIS supports the integration of database profiling into the agile development process. The key for such integration is the use of object databases, whose logical data model is equivalent to the application's conceptual data model. We demonstrated this using the ZooDB ODBMS, which has been extended to support profiling of queries and access paths, and AgileIS, which supports import of profiling data and subsequent semi-automatic refactoring based on refactoring suggestions provided by ZooDB. This type of database profiling closes the customer-feedback loop in the agile development cycle.

Compared to static analysis, database profiling has the disadvantage that the feedback is not immediately available. This is countered by the usually much greater advantage that the feedback can build on much richer data to identify problematic database usage and suggest more accurate refactorings. The suggested refactorings are mostly based on known refactorings and 'good practices' recommended by other authors, but we underpin them with actual data and can thus rank such refactorings with respect to expected benefit of their application.

While our approach may not always suggest conceptually sound refactorings, it generally yields useful information about potential mismatches between the conceptual model and the database usage, such as semantic modelling errors. We expect that, in many cases, an indication of these mismatches will result in useful suggestions for further investigation that may result in changes to the model or code and thus ultimately result in an improved development process.

There are several opportunities for future extensions to the AgileIS approach of database profiling. One interesting extension would be to apply database profiling to the database evolution process. The results could be used to investigate how model design affects the complexity of designing and executing database evolution code. A possible result would be general recommendations for patterns that should be avoided because they are hard or costly to refactor, or patterns that are recommended because they facilitate refactoring. Some such patterns have been suggested in [Section 4](#). Others could be the temporary duplication of attributes or data into multiple representations, for example in order to test alternative access patterns in real-life scenarios or to provide an easy fallback solution for complex refactorings.

A more complex improvement would be to calculate the cost–benefit ratio for combinations of refactoring patterns. As mentioned before, such calculations can be exponentially expensive if every possible combination of possible refactorings would be calculated. A first step would be to only calculate combinations of refactorings that have been preselected by the model designer. A second, more expensive, step could be to only pre-calculate combinations of refactorings that have a high cost–benefit ratio and that do not contradict each other, because such refactorings are likely to be chosen by the model designer. Also, calculating combinations of refactorings that are not related is probably not very useful. Finally, it may be possible to use more advanced algorithms for calculating the cost–benefit ratio, so that all combinations with a cost–benefit ratio over a given threshold are provided.

As discussed in [Section 6](#), we could observe that the suggested refactorings, be they based on static analysis or profiling, often coincide with semantically correct and sensible refactorings. It would be useful to investigate this further in order to better exploit the flow of semantic information in projects and understand the implications when conflicting understandings interact on the database level.

9. Conclusion

Developing information systems with iterative agile development practices poses unique challenges and opportunities. One challenge is the possibly increased effort of evolving databases. To approach this problem, we introduced the concept of evolvability of a data model and gave some indicators how it could be measured using static model analysis and how a data model could be refactored to improve evolvability.

Then we looked at database profiling which is traditionally used to improve application performance by optimising the database schema which represents the logical data model. With ODBMSs, however, the schema represents not only the logical data model but also the conceptual data model. This presents a number of opportunities for improving information system development.

One advantage of ODBMSs is that object-oriented applications tend to use a lot more navigation than queries to access their data model. Profiling this navigation in the database naturally provides a larger context to individual attribute and object access than just recording queries. Hence this context allows the generation of rich, accurate, and sensible suggestions for model refactorings.

We showed that the agile development cycle can be successfully extended with profiling feedback that is applied to the conceptual model. We also demonstrated that applying profiling data to the conceptual model does not necessarily 'pollute' the conceptual model with technical aspects of the physical or logical representation of the model in the database. To the contrary, profiling on object databases provides feedback that is not only richer than feedback from traditional profiling, but also makes sense on the conceptual level, to the point where it indicates semantic problems in the conceptual model.

Building on these results, we discussed how database profiling might be used to correlate an end-user's semantic understanding of the problem domain with the model designers' and developers' semantic understanding. This may open up current approaches to agile development and related research in two ways. First, database profiling can be seen as an additional tool for quality assurance of conceptual models. Second, the lack of database normalisation in real-world databases [\[30\]](#) may be seen not as a problem but rather as an advantage, because it seems likely that this denormalisation enables the semantic feedback loop which provides indicators for useful conceptual refactorings and diagnostics.

Acknowledgments

This research was partially funded by the Hasler Foundation, Switzerland.

References

- [1] M. Fowler, J. Highsmith, The Agile Manifesto, *Software, Development* 9 (8) (2001).
- [2] T. Zäschke, M.C. Norrie, Revisiting Schema Evolution in Object Databases in Support of Agile Development, *Proc. Intl. Conf. on Object Databases (ICOODB 2010)*, Springer Berlin / Heidelberg 2010, pp. 10–24.
- [3] S.W. Ambler, Agile Techniques for Object Databases, <http://www.db4o.com/about/productinformation/whitepapers/September 2005>.
- [4] T. Zäschke, Agile Software Development with Object Databases (Ph.D. thesis) ETH Zurich, 2014.
- [5] M. Ley, et al., The DBLP Computer Science Bibliography, <http://dblp.uni-trier.de/db/Mar. 2013>.
- [6] T. Zäschke, S. Leone, T. Gmünder, M.C. Norrie, Optimising Conceptual Data Models through Profiling in Object Databases, *Proc. 32st Intl. Conf. on Conceptual Modeling (ER 2013)* 2013, pp. 284–297.
- [7] S.W. Ambler, P.J. Sadalage, *Refactoring Databases: Evolutionary Database Design*, Addison-Wesley, 2006.
- [8] S. Chaudhuri, V. Narasayya, Self-Tuning Database Systems: A Decade of Progress, *Proc. 33rd Intl. Conf. on Very Large Databases (VLDB 2007)* 2007, pp. 3–14.
- [9] E.F. Codd, A Relational Model of Data for Large Shared Data Banks, *Commun. ACM* 13 (6) (1970) 377–387.
- [10] Y. Pinto, A framework for systematic database denormalization, *Glob. J. Comput. Sci. Technol.* 9 (4) (2009) 44–52.
- [11] G. Sanders, S. Shin, Denormalization Effects on Performance of RDBMS, *Proc. 34th Hawaii Intl. Conf. on System Sciences (HICSS 2001)* 2001, p. 9.
- [12] H.-J. Schek, P. Pistor, Data Structures for an Integrated Data Base Management and Information Retrieval System, *Proc. 8th Intl. Conf. on Very Large Databases (VLDB 1982)*, vol. 82 1982, pp. 197–207.
- [13] Objectivity, Objectivity/DB, <http://www.objectivity.com/2013>.
- [14] D.L. Moody, Theoretical and practical issues in evaluating the quality of conceptual models: current state and future directions, *Data Knowl. Eng.* 55 (3) (2005) 243–276.
- [15] H.A. Proper, T.A. Halpin, Conceptual Schema Optimisation—Database Optimisation Before Sliding Down the Waterfall, Tech. rep., Department of Computer Science, University of Queensland, Brisbane, Australia, 2004.
- [16] D.S. Cruzes, A. Vennesland, M.K. Natvig, Empirical Evaluation of the Quality of Conceptual Models Based on User Perceptions: A Case Study in the Transport Domain, *Proc. Intl. Conf. on Conceptual Modeling (ER 2013)*, vol. 8217 2013, pp. 414–428.
- [17] R. Fagin, *Armstrong Databases*, Citeseer, 1982.
- [18] V.B.T. Le, S. Link, F. Ferrarotti, Effective Recognition and Visualization of Semantic Requirements by Perfect SQL Samples, *Proc. Intl. Conf. on Conceptual Modeling (ER 2013)*, vol. 8217 2013, pp. 227–240.
- [19] D. Aguilera, C. Gómez, A. Olivé, A Method for the Definition and Treatment of Conceptual Schema Quality Issues, *Proc. Intl. Conf. on Conceptual Modeling (ER 2012)*, Springer 2012, pp. 501–514.
- [20] M. Fowler, et al., *Refactoring: Improving the Design of Existing Code*, Addison-Wesley, 1999.
- [21] P. Manousis, P. Vassiliadis, G. Papastefanatos, Automating the Adaptation of Evolving Data-Intensive Ecosystems, *Proc. Intl. Conf. on Conceptual Modeling (ER 2013)*, vol. 8217 2013, pp. 182–196.
- [22] T. Zäschke, et al., The ZooDB Object Database, <https://github.com/tzaeschke/zoodbSep 2014>.
- [23] T. Gmünder, Enhancing an Object Database with Profiling Information to Improve Agile Information Systems Development (Master's thesis) ETH Zurich, 2013.
- [24] T. Zäschke, C. Zimmerli, S. Leone, M. Nguyen, M.C. Norrie, Adaptive Model-Driven Information Systems Development for Object Databases, *Proc. Intl. Conf. on Information Systems Development (ISD 2011)* 2011, pp. 513–525.
- [25] T. Zäschke, S. Leone, M.C. Norrie, Optimising Schema Evolution Operation Sequences in Object Databases for Data Evolution, *Proc. 31st Intl. Conf. on Conceptual Modeling (ER 2012)* 2012, pp. 369–382.
- [26] Versant Corporation, *db4o Reference Documentation*, 2010.
- [27] Versant, Versant/JVI, <http://www.versant.com/2013>.
- [28] A. Davis, S. Overmyer, K. Jordan, J. Caruso, F. Dandashi, A. Dinh, G. Kincaid, G. Ledebor, P. Reynolds, P. Sitaram, A. Ta, M. Theofanos, Identifying and Measuring Quality in a Software Requirements Specification, *Software Metrics Symposium, 1993. Proceedings, First International 1993*, pp. 141–152.
- [29] U.ENZLER, Agile Quality Assurance URL <http://www.planetgeek.ch/2012/09/20/presentation-agile-quality-assurance/2012>.
- [30] A. Badia, D. Lemire, A call to arms: revisiting database design, *ACM SIGMOD Rec.* 40 (3) (2011) 61–69.
- [31] D. Maier, Representing Database Programs as Objects, *Proc. Intl. Workshop on Database Programming Languages (DBPL 1987)* 1987, pp. 377–386.
- [32] A.R. Hurson, S.H. Pakzad, J.-B. Cheng, Object-oriented database management systems: evolution and performance issues, *Computer* 26 (2) (1993) 48–58.
- [33] M.P. Atkinson, F. Bancelhon, D.J. DeWitt, K.R. Dittrich, D. Maier, S.B. Zdonik, The Object-Oriented Database System Manifesto, *Proc. 1st Intl. Conf. on Deductive and Object-Oriented Databases (DOOD)*, vol. 89 1989, pp. 40–57.